

Kiến thức tổng quát Apache Flink

Tài liệu này dùng để ôn phần kiến thức nền ngoài demo. Mục tiêu là giúp trả lời các câu hỏi phản biện rộng hơn về Apache Flink, Kafka, stream processing, state, checkpoint, watermark, window và khả năng scale hệ thống.

1. Apache Flink là gì?

Apache Flink là một distributed stream processing framework, dùng để xử lý dữ liệu liên tục theo thời gian thực hoặc gần thời gian thực.

Flink có thể xử lý:

- Unbounded stream: dữ liệu không có điểm kết thúc, ví dụ transaction, log, clickstream, IoT sensor.
- Bounded stream: dữ liệu có giới hạn, ví dụ file CSV, dữ liệu batch trong một khoảng thời gian.

Điểm quan trọng là Flink xem batch như một trường hợp đặc biệt của stream. Vì vậy cùng một engine có thể xử lý cả batch và streaming.

2. Stream processing khác batch processing như thế nào?

Batch processing xử lý dữ liệu theo từng lô lớn, thường có sẵn trước khi job chạy. Ví dụ: cuối ngày tổng hợp doanh thu của 24 giờ trước.

Stream processing xử lý dữ liệu khi nó vừa phát sinh. Ví dụ: khi transaction xuất hiện, hệ thống đọc ngay, tính toán ngay và có thể cảnh báo gần như tức thời.

So sánh ngắn:

Tiêu chí	Batch processing	Stream processing
Dữ liệu	Có giới hạn	Liên tục, không giới hạn
Thời điểm xử lý	Theo lịch, theo lô	Gần real-time
Độ trễ	Phút, giờ, ngày	Mili giây đến vài giây
Ví dụ	Báo cáo cuối ngày	Fraud detection, realtime dashboard

3. Vì sao cần Flink?

Flink phù hợp khi hệ thống cần:

- Xử lý dữ liệu liên tục với độ trễ thấp.
- Tính toán theo cửa sổ thời gian.
- Duy trì state trong quá trình xử lý.
- Đảm bảo fault tolerance bằng checkpoint.
- Scale ra nhiều máy.

- Xử lý event-time, dữ liệu đến trễ và dữ liệu out-of-order.

Ví dụ trong transaction analytics, Flink có thể tính tổng tiền theo account trong mỗi 10 giây, phát hiện account có nhiều giao dịch bất thường, hoặc cập nhật dashboard realtime.

4. Kiến trúc tổng quát của Flink

Một Flink cluster thường có:

- JobManager
- TaskManager
- Client

JobManager

JobManager chịu trách nhiệm điều phối:

- Nhận job từ client.
- Chuyển chương trình thành execution graph.
- Lập lịch các task.
- Điều phối checkpoint.
- Theo dõi trạng thái job.
- Khôi phục job khi có lỗi.

JobManager xử lý control flow, không phải nơi dữ liệu transaction đi qua.

TaskManager

TaskManager là nơi thực thi xử lý dữ liệu thật sự.

TaskManager chịu trách nhiệm:

- Chạy các subtasks.
- Nhận dữ liệu từ source.
- Xử lý operator như map, filter, keyBy, window, aggregate.
- Gửi dữ liệu sang sink.
- Lưu và quản lý state cục bộ.

Client

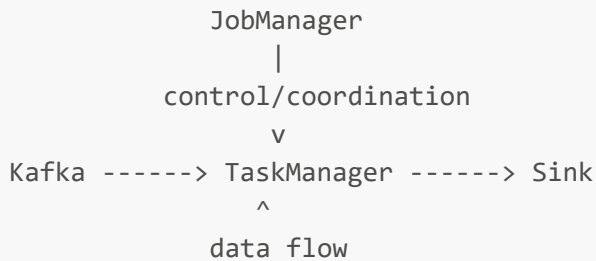
Client là nơi submit job lên Flink cluster. Sau khi submit xong, client không nhất thiết phải tiếp tục chạy để job tồn tại.

5. Data flow và control flow khác nhau thế nào?

Control flow là luồng điều phối, do JobManager quản lý.

Data flow là luồng dữ liệu thật, thường đi trực tiếp từ source đến TaskManager rồi đến sink.

Sơ đồ đúng:



Không nên nói rằng transaction đi theo luồng:

```
Kafka -> JobManager -> TaskManager
```

Vì JobManager không xử lý từng transaction.

6. Operator là gì?

Operator là một bước xử lý trong Flink job.

Ví dụ:

- Source: đọc dữ liệu từ Kafka.
- Map: biến đổi dữ liệu.
- Filter: lọc dữ liệu.
- KeyBy: chia dữ liệu theo key.
- Window: gom dữ liệu theo thời gian.
- Aggregate: tính tổng, count, average.
- Sink: ghi kết quả ra ngoài.

Một pipeline đơn giản:

```
Kafka Source -> Parse -> Filter -> keyBy -> Window -> Aggregate -> Sink
```

7. Parallelism là gì?

Parallelism là số instance song song của một operator.

Nếu một operator có parallelism = 2, operator đó có 2 subtasks chạy song song.

Ví dụ:

```
Source parallelism = 2
```

```
Source Subtask 0
Source Subtask 1
```

Parallelism càng cao thì hệ thống có khả năng xử lý song song nhiều hơn, nhưng chỉ hiệu quả nếu nguồn dữ liệu, tài nguyên CPU/memory và downstream operator cũng đáp ứng được.

8. Subtask là gì?

Subtask là một instance chạy song song của operator.

Ví dụ operator **Aggregate** có parallelism = 2 thì sẽ có:

```
Aggregate Subtask 0
Aggregate Subtask 1
```

Mỗi subtask xử lý một phần dữ liệu.

9. Task slot là gì?

Task slot là đơn vị tài nguyên thực thi trong TaskManager.

Task slot không tương đương với Kafka partition. Một slot có thể chạy một chuỗi operator subtasks nếu Flink chain các operator lại với nhau.

Quan hệ cần nhớ:

- Partition thuộc Kafka.
 - Subtask thuộc Flink operator.
 - Task slot thuộc tài nguyên thực thi của TaskManager.
-

10. Kafka đóng vai trò gì trong hệ thống Flink?

Kafka thường đóng vai trò là source hoặc message broker ở trước Flink.

Kafka giúp:

- Nhận dữ liệu từ producer.
- Lưu dữ liệu theo topic/partition.
- Buffer khi Flink xử lý chậm.
- Cho phép Flink đọc lại dữ liệu sau khi restart.
- Hỗ trợ nhiều consumer group cùng đọc một topic.
- Tăng khả năng scale bằng partition.

Kafka không thay thế Flink, vì Kafka chủ yếu lưu và truyền dữ liệu. Flink mới là nơi xử lý logic tính toán.

11. Kafka topic và partition là gì?

Topic là luồng dữ liệu logic trong Kafka. Ví dụ `transactions`.

Partition là cách Kafka chia nhỏ topic để lưu trữ và xử lý song song.

Ví dụ topic `transactions` có 3 partitions:

```
transactions
├── partition 0
├── partition 1
└── partition 2
```

Một partition có thứ tự nội bộ. Kafka đảm bảo order trong cùng một partition, nhưng không đảm bảo order tuyệt đối giữa các partitions khác nhau.

12. Kafka offset là gì?

Offset là vị trí của record trong một Kafka partition.

Ví dụ:

```
partition 0:
offset 0, offset 1, offset 2, ...
```

Flink dùng offset để biết đã đọc đến đâu. Khi checkpoint, Flink lưu offset cùng với state để khi restart có thể đọc lại từ vị trí nhất quán.

13. keyBy là gì?

`keyBy` dùng để chia stream theo key.

Ví dụ:

```
keyBy(account_id)
```

Nghĩa là tất cả transaction có cùng `account_id` sẽ được đưa về cùng một downstream subtask. Điều này giúp Flink tính state/window riêng cho từng account.

14. Vì sao keyBy tạo ra HASH trong Job Graph?

Khi dùng `keyBy`, Flink hash giá trị key để quyết định record đi về subtask nào.

Ví dụ:

```
hash(account_id) -> chọn downstream subtask
```

Vì vậy trong Job Graph có thể thấy cạnh dữ liệu kiểu **HASH**.

15. State là gì?

State là dữ liệu trung gian mà Flink phải nhớ trong quá trình xử lý.

Ví dụ state:

- Tổng tiền hiện tại của account trong window.
- Số lượng transaction đã nhận.
- Danh sách event đang chờ xử lý.
- Offset của source.
- Trạng thái operator.

Nếu không có state, Flink chỉ xử lý từng record riêng lẻ và rất khó làm các bài toán như window, aggregate, join, fraud detection.

16. Keyed state và operator state khác nhau thế nào?

Keyed state là state gắn với từng key sau khi **keyBy**.

Ví dụ:

```
acc-01 -> total = 900  
acc-02 -> total = 300
```

Operator state là state gắn với một operator subtask, không chia theo key cụ thể. Ví dụ Kafka source có thể lưu thông tin partitions/offsets mà subtask đang đọc.

17. Window là gì?

Window là cơ chế chia stream vô hạn thành các đoạn hữu hạn để tính toán.

Vì stream không có điểm kết thúc, ta không thể chờ "hết dữ liệu" rồi mới aggregate. Window giúp đặt ranh giới, ví dụ tính tổng transaction mỗi 10 giây.

Ví dụ:

```
10:00:00 - 10:00:10  
10:00:10 - 10:00:20  
10:00:20 - 10:00:30
```

18. Tumbling window là gì?

Tumbling window là window không chồng lấn.

Ví dụ window 10 giây:

```
[00s - 10s)
[10s - 20s)
[20s - 30s)
```

Mỗi event chỉ thuộc một window.

19. Sliding window là gì?

Sliding window là window có thể chồng lấn.

Ví dụ size = 10 giây, slide = 5 giây:

```
[00s - 10s)
[05s - 15s)
[10s - 20s)
```

Một event có thể thuộc nhiều window. Nếu size 10 giây và slide 5 giây, một event thường thuộc 2 windows.

20. Session window là gì?

Session window gom các event theo phiên hoạt động, dựa trên khoảng im lặng giữa các event.

Ví dụ session gap = 30 giây:

- Nếu hai event cách nhau dưới 30 giây, chúng có thể thuộc cùng một session.
- Nếu sau 30 giây không có event mới, session đóng lại.

Session window phù hợp với bài toán hành vi người dùng, clickstream, phiên đăng nhập hoặc chuỗi giao dịch gần nhau.

21. Processing Time và Event Time khác nhau thế nào?

Processing Time là thời gian khi Flink xử lý event.

Event Time là thời gian thật sự event xảy ra, thường nằm trong dữ liệu, ví dụ trường `event_time`.

Ví dụ transaction xảy ra lúc 10:00:05 nhưng đến Flink lúc 10:00:20:

- Processing Time: event thuộc window quanh 10:00:20.

- Event Time: event thuộc window quanh 10:00:05.

Event Time chính xác hơn cho dữ liệu thực tế vì dữ liệu có thể đến trễ hoặc lệch thứ tự.

22. Watermark là gì?

Watermark là cơ chế giúp Flink xử lý Event Time.

Watermark báo rằng Flink tin rằng các event có timestamp nhỏ hơn hoặc bằng một mốc thời gian nào đó phần lớn đã đến. Khi watermark vượt qua cuối window, Flink có thể trigger window và xuất kết quả.

Ví dụ:

```
Window: [10:00:00 - 10:00:10)
Watermark: 10:00:10
```

Khi watermark đạt hoặc vượt 10:00:10, window này có thể được đóng và tính kết quả.

23. Late event là gì?

Late event là event đến sau khi watermark đã vượt qua window mà event đó thuộc về.

Ví dụ event có `event_time = 10:00:05`, nhưng watermark đã là `10:00:20`. Khi đó event này bị xem là đến trễ.

Cách xử lý late event:

- Drop event trễ.
 - Cho phép trễ bằng `allowed lateness`.
 - Gửi event trễ sang side output.
 - Cập nhật lại kết quả nếu sink hỗ trợ update/upsert.
-

24. Checkpoint là gì?

Checkpoint là snapshot định kỳ của trạng thái Flink job để phục hồi khi có lỗi.

Checkpoint lưu:

- Kafka offsets/source progress.
- Keyed state.
- Operator state.
- Window state.
- Metadata cần để restore nhất quán.

Checkpoint không lưu toàn bộ dữ liệu Kafka. Kafka vẫn là nơi giữ input records.

25. Checkpoint khác savepoint như thế nào?

Checkpoint thường được Flink tạo tự động theo chu kỳ để fault tolerance. Khi job fail, Flink dùng checkpoint gần nhất để khôi phục.

Savepoint thường được tạo thủ công để phục vụ thao tác vận hành như:

- Dừng job có kiểm soát.
- Nâng cấp version code.
- Thay đổi cấu hình.
- Di chuyển job sang cluster khác.

Nói ngắn:

- Checkpoint: phục hồi lỗi tự động.
- Savepoint: điểm lưu thủ công để bảo trì, nâng cấp, migrate.

26. Exactly-once nghĩa là gì?

Exactly-once nghĩa là kết quả cuối cùng của hệ thống tương đương với việc mỗi record được xử lý đúng một lần, kể cả khi có lỗi và restart.

Quan trọng: exactly-once không chỉ phụ thuộc vào Flink. Nó phụ thuộc vào cả pipeline:

```
Source -> Flink state -> Sink
```

Nếu source và Flink hỗ trợ checkpoint nhưng sink không hỗ trợ transactional/idempotent write, kết quả bên ngoài vẫn có thể bị duplicate.

27. At-least-once và at-most-once khác nhau thế nào?

At-most-once: mỗi record được xử lý tối đa một lần. Có thể mất dữ liệu khi lỗi.

At-least-once: mỗi record được xử lý ít nhất một lần. Không mất dữ liệu, nhưng có thể xử lý trùng.

Exactly-once: kết quả cuối cùng tương đương xử lý đúng một lần.

So sánh:

Semantics	Có thể mất dữ liệu	Có thể duplicate
At-most-once	Có	Không hoặc ít
At-least-once	Không	Có
Exactly-once	Không	Không ở kết quả cuối, nếu toàn pipeline hỗ trợ

28. Sink cần gì để exactly-once?

Sink cần có một trong các khả năng:

- Transactional write.
- Two-phase commit.
- Idempotent write.
- Upsert theo khóa duy nhất.

Ví dụ database sink nên ghi theo key như:

```
(account_id, window_start, window_end)
```

Nếu job xử lý lại cùng một window, database update/overwrite kết quả cũ thay vì insert thêm dòng trùng.

29. Backpressure là gì?

Backpressure xảy ra khi downstream operator hoặc sink xử lý chậm hơn upstream.

Ví dụ Kafka source đọc nhanh, nhưng database sink ghi chậm. Khi đó dữ liệu bị dồn lại, các operator phía trước phải chậm theo.

Dấu hiệu:

- Throughput giảm.
- Checkpoint lâu hơn.
- Task bận cao.
- Buffer đầy.
- Latency tăng.

Cách xử lý:

- Tăng parallelism.
 - Tối ưu operator chậm.
 - Tối ưu sink.
 - Batch write ở sink.
 - Tăng tài nguyên CPU/memory.
 - Kiểm tra data skew/hot key.
-

30. Data skew và hot key là gì?

Data skew là dữ liệu phân phối không đều giữa các subtasks.

Hot key là một key có lượng dữ liệu quá lớn so với các key khác.

Ví dụ `acc-01` chiếm 80% transaction. Khi `keyBy(account_id)`, toàn bộ transaction của `acc-01` phải về cùng một subtask, làm subtask đó quá tải.

Flink không tự động chia một key sang nhiều subtasks vì như vậy sẽ phá vỡ tính đúng đắn của keyed state.

Cách xử lý:

- Thêm key phụ để chia tải.
 - Pre-aggregate rồi aggregate lại.
 - Tách riêng hot key sang pipeline riêng.
 - Dùng chiến lược partitioning phù hợp.
 - Tối ưu logic xử lý cho hot key.
-

31. Scale Flink cần chú ý gì?

Khi scale Flink, cần xem:

- Số Kafka partitions.
- Parallelism của source.
- Parallelism của downstream operators.
- Số TaskManagers.
- Tổng số task slots.
- CPU, memory, network.
- State backend và checkpoint storage.
- Sink có chịu được throughput tăng không.

Không phải cứ tăng parallelism là throughput tăng. Nếu Kafka chỉ có 2 partitions, source parallelism 10 vẫn chỉ có tối đa 2 subtasks đọc dữ liệu thật sự.

32. Có cần sửa logic khi scale từ laptop lên cluster không?

Thường là không cần sửa logic nghiệp vụ.

Logic có thể giữ nguyên:

```
Kafka -> keyBy -> window -> aggregate -> sink
```

Nhưng physical deployment thay đổi:

```
Laptop:  
1 TaskManager  
  
Cluster:  
Machine A -> TaskManager  
Machine B -> TaskManager  
Machine C -> TaskManager  
Machine D -> TaskManager
```

Khi scale, cần tăng partitions, parallelism và tài nguyên phù hợp. Chỉ cần sửa logic nếu gặp vấn đề như hot key, sink bottleneck, hoặc yêu cầu nghiệp vụ mới.

33. State backend là gì?

State backend quyết định Flink lưu state ở đâu và lưu như thế nào trong lúc job chạy.

Một số backend phổ biến:

- `HashMapStateBackend`: state nằm trong memory của `TaskManager`, checkpoint ra storage ngoài.
- `EmbeddedRocksDBStateBackend`: state nằm trong `RocksDB` local, phù hợp state lớn hơn memory.

Với state lớn, `RocksDB` thường phù hợp hơn nhưng có thể chậm hơn memory.

34. Checkpoint storage là gì?

Checkpoint storage là nơi lưu checkpoint bền vững.

Ví dụ:

- Local filesystem cho demo.
- HDFS.
- S3.
- MinIO.
- Cloud object storage.

Trong production, không nên chỉ lưu checkpoint ở local disk tạm thời của một máy, vì nếu máy mất thì checkpoint cũng mất.

35. Flink xử lý lỗi như thế nào?

Khi một task fail, Flink có thể restart job hoặc một vùng của job tùy restart strategy và failover strategy.

Quá trình khôi phục thường là:

1. Phát hiện lỗi.
2. Dừng các task liên quan.
3. Restore state từ checkpoint gần nhất.
4. Source đọc lại từ offset đã checkpoint.
5. Job tiếp tục xử lý.

Nhờ Kafka replay và Flink checkpoint, hệ thống có thể phục hồi mà không mất dữ liệu nếu cấu hình đúng.

36. Operator chaining là gì?

Operator chaining là việc Flink gộp nhiều operator liên tiếp vào cùng một task để giảm overhead truyền dữ liệu.

Ví dụ:

```
Source -> Map -> Filter
```

có thể được chain lại nếu phù hợp.

Lợi ích:

- Giảm serialization/deserialization.
- Giảm network shuffle.
- Tăng hiệu năng.

Nhưng khi có **keyBy**, dữ liệu phải shuffle theo key nên thường tạo ranh giới giữa các task/operator chain.

37. Khi nào có network shuffle?

Network shuffle xảy ra khi dữ liệu cần chuyển giữa các subtasks, đặc biệt khi:

- **keyBy**
- **rebalance**
- **rescale**
- Thay đổi parallelism giữa upstream và downstream

Với **keyBy**, Flink phải gửi record đến đúng subtask sở hữu key đó, nên có thể phát sinh shuffle qua network.

38. Flink khác Spark Streaming ở điểm nào?

Flink là stream-first, xử lý từng event hoặc từng record theo luồng liên tục.

Spark truyền thống thiên về batch-first, Spark Structured Streaming dùng mô hình micro-batch trong nhiều trường hợp.

Nói ngắn:

- Flink mạnh về low-latency true streaming, event time, stateful stream processing.
- Spark mạnh về hệ sinh thái batch, SQL, ML và tích hợp dữ liệu lớn.

Cách trả lời nên tránh tuyệt đối hóa. Không nên nói "Flink luôn tốt hơn Spark". Nên nói tùy bài toán.

39. Flink có thay thế Kafka không?

Không. Kafka và Flink giải quyết hai lớp khác nhau.

Kafka:

- Lưu và truyền stream dữ liệu.
- Quản lý topic, partition, offset, retention.

Flink:

- Xử lý stream dữ liệu.
- Tính toán stateful, window, aggregate, join, detect pattern.

Trong nhiều hệ thống realtime, Kafka và Flink bổ sung cho nhau.

40. Câu trả lời ngắn khi bị hỏi "điểm mạnh nhất của Flink là gì?"

Điểm mạnh nhất của Flink là xử lý stream có state với độ trễ thấp, hỗ trợ event time, watermark, checkpoint và khả năng scale phân tán. Vì vậy Flink phù hợp với các bài toán realtime như transaction analytics, fraud detection, realtime dashboard và monitoring.

41. Flink có những API layer nào?

Flink có nhiều mức API, từ dễ dùng đến kiểm soát sâu:

- Flink SQL: viết truy vấn SQL theo chuẩn gần ANSI, phù hợp phân tích dữ liệu, dashboard, ETL.
- Table API: API dạng lập trình nhưng vẫn ở mức khai báo cao.
- DataStream API: xử lý stream bằng các operator như map, filter, keyBy, window, aggregate.
- ProcessFunction: API mức thấp, cho phép truy cập state, timer, event time và side output.

Nói ngắn: SQL/Table API phù hợp khi muốn khai báo logic; DataStream API phù hợp khi cần xử lý stream linh hoạt; ProcessFunction phù hợp khi cần kiểm soát chi tiết.

42. Flink SQL và Table API dùng để làm gì?

Flink SQL và Table API là high-level API. Người dùng mô tả muốn tính gì, còn Flink tối ưu cách chạy.

Các thành phần liên quan:

- Catalog: lưu schema, table definition và metadata.
- Connector: đọc/ghi dữ liệu từ Kafka, filesystem, database, Elasticsearch...
- Built-in functions: hỗ trợ window, aggregate, time function...
- Calcite Optimizer: parse, validate và tối ưu query trước khi chuyển thành execution plan.

Ví dụ có thể dùng SQL để tính tổng transaction theo window mà không cần viết nhiều code DataStream.

43. DataStream API dùng khi nào?

DataStream API dùng khi cần lập trình trực tiếp pipeline xử lý stream.

Các object/operator thường gặp:

- **DataStream**: luồng dữ liệu chính.
- **KeyedStream**: luồng sau khi **keyBy**, dữ liệu được chia theo key.
- **map**, **filter**, **flatMap**: biến đổi dữ liệu.
- **window**: gom event theo cửa sổ thời gian.
- **aggregate**: tính toán kết quả.
- **connector**: đọc từ source và ghi ra sink.

DataStream API phù hợp với demo Kafka -> Flink -> analytics vì dễ thể hiện rõ source, transformation, keyBy, window và sink.

44. ProcessFunction dùng khi nào?

ProcessFunction là low-level API trong Flink. Nó xử lý từng event và cho phép truy cập nhiều cơ chế nội bộ hơn các operator thông thường.

ProcessFunction hữu ích khi cần:

- Dùng keyed state thủ công như **ValueState**, **MapState**.
- Đăng ký timer bằng **TimerService**.
- Xử lý theo event time hoặc processing time ở mức chi tiết.
- Phát dữ liệu ra nhiều nhánh bằng side output.
- Xây dựng state machine hoặc logic cảnh báo phức tạp.

Nếu bài toán chỉ là window aggregate đơn giản thì không nhất thiết cần ProcessFunction.

45. Dispatcher và ResourceManager trong Flink là gì?

Ngoài JobManager và TaskManager, kiến trúc Flink còn có Dispatcher và ResourceManager.

Dispatcher nhận job được submit và khởi tạo JobManager cho job đó trong một số deployment mode.

ResourceManager quản lý tài nguyên của cluster, đặc biệt là task slots. Nó làm việc với TaskManagers để cấp slot cho job.

Tóm tắt:

- Dispatcher: nhận và khởi chạy job.
 - ResourceManager: quản lý slot/tài nguyên.
 - JobManager: điều phối execution của job.
 - TaskManager: chạy subtasks và xử lý dữ liệu thật.
-

46. Flink có thể deploy ở đâu?

Flink có thể chạy trên nhiều môi trường:

- Standalone cluster.
- YARN.
- Kubernetes.
- Cloud managed service, ví dụ Amazon Managed Service for Apache Flink.
- Platform thương mại như Ververica Platform.

Điểm cần nhớ: runtime và logic Flink không đổi, nhưng môi trường deployment quyết định cách cấp tài nguyên, quản lý container, scaling, monitoring và vận hành.

47. Chi phí triển khai Flink gồm những gì?

Với self-managed Flink, chi phí chính là:

- Máy chủ hoặc Kubernetes cluster.
- Compute, storage, network.
- Checkpoint storage như HDFS, S3, MinIO.
- Monitoring, logging, alerting.
- Công sức vận hành, tuning, xử lý lỗi.

Với managed service hoặc platform thương mại, chi phí hạ tầng và vận hành giảm bớt nhưng có thêm phí dịch vụ, license hoặc tính tiền theo tài nguyên sử dụng.

48. Flink, Spark Structured Streaming và Kafka Streams khác nhau thế nào?

So sánh ngắn:

Tiêu chí	Apache Flink	Spark Structured Streaming	Kafka Streams
Mô hình	True streaming	Thường là micro-batch	Library gắn với app Kafka
Độ trễ	Rất thấp	Thường cao hơn Flink	Thấp, phụ thuộc app
State	Mạnh, phù hợp state lớn/dài hạn	Tốt, mạnh trong hệ Spark	Tốt cho state vừa phải
Triển khai	Cluster engine riêng	Spark cluster	Java app, không cần cluster riêng
Nguồn dữ liệu	Nhiều connector	Mạnh trong hệ Spark	Chủ yếu Kafka
Phù hợp	Realtime pipeline phức tạp	Tổ chức đã dùng Spark nhiều	App xử lý stream Kafka đơn giản/vừa

Không nên nói công cụ nào luôn tốt hơn. Chọn công cụ tùy yêu cầu latency, state, hệ sinh thái, đội ngũ và chi phí vận hành.

49. Savepoint claim mode là gì?

Khi restore từ savepoint, Flink có các mode liên quan đến quyền sở hữu file savepoint.

Nói ở mức tổng quát:

- **NO_CLAIM**: Flink không nhận quyền sở hữu savepoint gốc, an toàn hơn khi muốn giữ savepoint để dùng lại.
- **CLAIM**: Flink có thể nhận quyền quản lý một phần tài nguyên liên quan, phù hợp khi muốn tiếp tục vận hành từ savepoint đó theo cách tích hợp hơn.

Khi đi seminar, chỉ cần nắm ý chính: savepoint là điểm lưu thủ công để upgrade/migrate, còn claim mode liên quan đến cách Flink quản lý tài nguyên savepoint sau khi restore.

50. Các lỗi trả lời nên tránh

Không nên nói:

- Kafka xử lý dữ liệu thay Flink.
- JobManager là nơi transaction đi qua.
- Mỗi Kafka partition cần một task slot.
- Tăng parallelism luôn làm throughput tăng.
- `enableCheckpointing()` là đủ để exactly-once toàn hệ thống.
- `keyBy` tự động chia một hot key sang nhiều subtasks.
- Có 1 triệu key thì cần 1 triệu subtasks.
- Window state không phải state.
- Processing Time và Event Time giống nhau.
- Flink SQL/Table API, DataStream API và ProcessFunction là cùng một mức abstraction.
- Kafka Streams cần một Flink cluster để chạy.

51. Mẫu trả lời tổng kết khi thầy hỏi rộng

Nếu thầy hỏi tổng quát về kiến trúc, có thể trả lời:

Trong hệ thống của em, Kafka đóng vai trò lưu và phân phối transaction stream, còn Flink là engine xử lý realtime. Producer ghi dữ liệu vào Kafka topic, Flink Kafka Source đọc dữ liệu theo offset, sau đó parse/filter, `keyBy(account_id)`, gom theo window và aggregate. JobManager điều phối job và checkpoint, còn TaskManager mới là nơi dữ liệu thực sự được xử lý. Khi có lỗi, Flink restore state và Kafka offsets từ checkpoint gần nhất, rồi đọc lại dữ liệu từ Kafka để đảm bảo phục hồi nhất quán. Nếu muốn scale, em tăng Kafka partitions, Flink parallelism, task slots và số TaskManagers, còn logic nghiệp vụ `keyBy -> window -> aggregate` thường không cần thay đổi.